Kinetix AI: An AI-Powered Sports Analytics Platform for Cricket Performance Analysis and Match Insights Using MERN Stack and Machine Learning

Dikshant Patel, Rana Dishant Vipul, Fenil Vachhani, Patel Sahil
Department of Information and Technology
Enrollment No: 2303031080038, 2303031080243, 2303031080253, 2303031080236

Abstract

The rapid growth of digital cricket data, ranging from ball-by-ball commentary to biometric player tracking, has created a strong demand for platforms that can convert raw statistics into actionable insight. This paper presents Kinetix AI, an AI-powered sports analytics platform designed to analyse cricket performance and generate match insights using the MERN stack (MongoDB, Express.js, React, Node.js) combined with machine learning models. The platform ingests live and historical match data through public and third-party cricket APIs, processes it through a Node.js backend, stores structured and semi-structured records in MongoDB, and exposes RESTful endpoints consumed by a React-based dashboard. Machine learning components include a win-probability predictor built on gradient-boosted trees, a player-form estimator using time-series regression, and a rule-assisted fatigue/injury-risk indicator derived from workload metrics. The system also provides interactive visual dashboards for coaches, analysts, and fans, covering batting and bowling performance, strike-rate trends, and match-progression charts. Evaluation on a held-out set of matches shows that the win-probability model achieves competitive accuracy compared to baseline heuristic models, while the dashboard reduces the time needed to compile a post-match performance summary. The paper details the system architecture, database design, module decomposition, implementation choices, and evaluation results, and discusses the advantages, limitations, and future scope of the platform, including planned support for additional formats of the game and deeper injury-prevention analytics.

Keywords—*Sports Analytics, Cricket Performance Analysis, MERN Stack, Machine Learning, Win Probability Prediction, Player Performance Dashboard, Data Visualization, REST API.*

I. INTRODUCTION

Cricket generates an unusually large volume of structured event data for every delivery bowled, including runs scored, dismissal type, field placement, and match situation. Historically, this data has been used mainly for retrospective scorecards and broadcast graphics, with detailed analytical interpretation left to specialist commentators or team analysts working with spreadsheets and proprietary tools. In recent years, the availability of open cricket APIs and the maturity of web application frameworks have made it feasible to build accessible, low-cost analytics platforms that were previously the preserve of professional franchises with dedicated data-science teams.

Kinetix AI is proposed as a full-stack sports analytics platform that focuses specifically on cricket. It is built using the MERN stack, a widely adopted combination of MongoDB, Express.js, React, and Node.js that allows a single language, JavaScript, to be used across the client, server, and, indirectly, the data layer. The platform is designed around three pillars: (i) reliable ingestion and normalisation of live and historical match data, (ii) machine-learning-driven insight generation such as win-probability estimation and player-form tracking, and (iii) an intuitive, responsive dashboard that presents these insights to coaches, analysts, and enthusiasts without requiring any statistical background.

The motivation for this project stems from a gap observed between the depth of data now publicly available and the depth of insight that casual users, students, and smaller cricket organisations can practically extract from it. Existing solutions are frequently either broadcast-oriented (rich visuals, but not exportable or queryable), or heavily technical (raw datasets requiring data-science expertise), leaving a middle ground underserved. Kinetix AI aims to occupy that middle ground by combining automated data pipelines with pre-built, ready-to-interpret analytical views.

Beyond the immediate goal of building a working prototype, the project is also motivated by pedagogical and practical considerations relevant to a final-year engineering submission. The system deliberately spans the full breadth of a modern web application stack, front-end presentation, back-end API design, database schema modelling, and applied machine learning, so that it can serve both as a demonstrable end-user product and as a vehicle for exercising the complete software-engineering lifecycle: requirements analysis, architecture design, implementation, testing, and evaluation. The choice of cricket as the target domain is deliberate as well: the sport's discrete, sequential event structure (delivery-by-delivery outcomes) makes it particularly well suited to both document-oriented storage and sequential/time-series predictive modelling, while its very large global following ensures that the resulting insights are of genuine interest to a wide audience of coaches, students, and fans alike.

The principal contributions of this work can be summarised as follows: (1) an end-to-end system architecture that integrates live data ingestion, document-oriented persistence, machine-learning inference, and interactive visualization within a single coherent MERN-based application; (2) a practical, relative-baseline formulation of bowling workload risk that avoids the pitfalls of fixed, one-size-fits-all thresholds; (3) a comparative evaluation of win-probability modelling approaches, from simple heuristics to gradient-boosted ensembles, on a consistent held-out dataset; and (4) a modular design, validated through the implementation itself, that is explicitly intended to extend beyond cricket to other sports with minimal structural change.

This paper is organised as follows. Section II states the problem being addressed. Section III reviews related work in sports analytics and performance-prediction systems. Section IV lists the objectives of the project. Section V describes the proposed methodology and overall system architecture. Section VI outlines the technologies used. Section VII presents the detailed system design, including the database schema and module decomposition. Section VIII discusses implementation details. Section IX reports results and discussion. Section X summarises advantages and limitations. Section XI outlines future scope, and Section XII concludes the paper.

II. PROBLEM STATEMENT

Coaches, team analysts, and cricket enthusiasts require timely, digestible insight from match data, but currently face three recurring difficulties. First, live data feeds from public APIs are typically raw and inconsistent in structure, requiring substantial manual cleaning before any analysis can begin. Second, converting raw ball-by-ball statistics into higher-order insights, such as the probability of a team winning at a given match state, or an early indicator of a player's declining form or physical workload, requires statistical or machine-learning expertise that most end users do not possess. Third, existing tools rarely combine data ingestion, analytical

modelling, and visual presentation into one accessible, extensible application; analysts often stitch together spreadsheets, one-off scripts, and separate charting tools, which is time-consuming and error-prone.

The problem addressed by this work is therefore: how can a single, extensible web platform ingest heterogeneous cricket data sources, apply machine-learning models to derive performance and outcome insights, and present these insights through a coherent, interactive dashboard, in a way that is maintainable, reasonably accurate, and usable by non-technical stakeholders?

This problem can be further decomposed into a set of concrete sub-problems that the system design must resolve. At the data layer, the platform must reconcile differences in field naming, update frequency, and completeness across whichever external API or APIs are used, while remaining resilient to partial outages or malformed responses during a live match. At the modelling layer, the platform must select features and algorithms that generalise across different match situations (chasing a target versus setting one, different grounds, different formats of the game) without requiring a separate bespoke model for every scenario. At the presentation layer, the system must communicate probabilistic and statistical outputs, which can be easy to misinterpret, in a way that is visually intuitive and does not overstate the certainty of any single prediction. Addressing all three sub-problems within one coherent codebase, rather than as isolated scripts, is itself a non-trivial software-engineering challenge that this project treats as a core part of the problem statement.

III. LITERATURE REVIEW

Sports analytics has been studied extensively across several sports, and cricket-specific analytics has grown as a distinct sub-area over the last decade. This section reviews representative directions in the literature and situates Kinetix AI relative to them. Broadly, prior work can be grouped into four themes: (a) outcome and win-probability prediction models, (b) player performance and form modelling, (c) workload and injury-risk estimation, and (d) analytics dashboards and visual sports-data systems.

A. Win-Probability and Outcome Prediction

Outcome-prediction studies typically model win probability as a function of match-state variables such as runs scored, wickets lost, overs remaining, and historical team strength, using logistic regression, decision trees, or ensemble methods such as random forests and gradient boosting. These approaches generally report that ensemble tree-based models outperform simple heuristic run-rate comparisons, particularly in the middle overs of a limited-overs innings where match state is most fluid and a simple linear extrapolation of the required run rate is least representative of the true chase difficulty. A recurring theme is that model calibration, not just raw classification accuracy, matters for this task, since a win-probability figure is typically displayed directly to end users and a poorly calibrated model can be actively misleading even if its overall accuracy appears acceptable.

B. Player Performance and Form Modelling

Player-performance modelling studies commonly use rolling averages, exponentially weighted moving statistics, or time-series models to track a player's recent form, motivated by the observation that career-long averages are poor predictors of near-term performance, particularly for players returning from injury, a change in conditions, or a long lay-off. Some studies additionally attempt to cluster players into batting or bowling archetypes based on shot selection or delivery-type distributions, which is useful for team-composition and opposition-scouting purposes but requires higher-resolution tracking data than the ball-by-ball outcome data used in the present work.

C. Workload and Injury-Risk Estimation

Workload and injury-risk studies in cricket and other sports borrow from sports-science literature on acute:chronic workload ratios, applying similar ideas to bowling workload in particular, given the well-documented link between bowling overload and stress-related injuries such as lumbar stress fractures. A consistent methodological point in this literature is that workload should be assessed relative to an individual player's own recent training and match history rather than against a single fixed threshold applied uniformly across all players, since tolerance to workload varies considerably between individuals.

D. Dashboards and Visualization Systems

Dashboard and visualization-oriented work emphasises the value of interactive, drill-down charts over static scorecards, arguing that visual literacy among coaches and fans is generally higher than statistical literacy, so well-designed charts communicate insight more effectively than raw numbers or model outputs presented in isolation. This literature also cautions against dashboard overload, that is, presenting too many simultaneous metrics without clear visual hierarchy, which can obscure rather than clarify the most decision-relevant insight for a given moment in a match.

Table I compares Kinetix AI with representative categories of existing systems and research directions along dimensions relevant to this project: data ingestion automation, use of machine learning for prediction, workload/injury awareness, and end-to-end dashboard delivery.

Ref.	Focus Area	ML-based?	Live Data
[1]	Win-probability modelling (limited-overs)	Yes	Partial
[2]	Team strength / outcome prediction	Yes	No
[3]	Player rolling-form estimation	Yes	No
[4]	Batting performance clustering	Yes	No
[5]	Bowling workload and injury risk	Partial	No
[6]	General sports-data visualization	No	Partial
[7]	Fantasy-sports player scoring	Yes	Partial
[8]	Broadcast graphics analytics	No	Yes
[9]	Web-based sports dashboards (general)	No	Partial
Kinetix AI (proposed)	End-to-end MERN platform	Yes	Yes

TABLE I. COMPARISON OF KINETIX AI WITH REPRESENTATIVE PRIOR WORK

As Table I indicates, individual prior efforts tend to focus on a single concern, either predictive modelling, workload/injury analysis, or visualization, rather than integrating all of them into a single deployable, data-

connected application. Kinetix AI's contribution is therefore less about proposing a fundamentally new algorithm and more about integrating established techniques from these separate threads into a coherent, extensible, full-stack platform with live data connectivity, which is a gap consistently visible across the reviewed systems.

A further observation from the reviewed literature is methodological: studies that treat win-probability estimation as a supervised learning problem over match-state snapshots consistently report that ensemble tree methods (random forests, gradient boosting) outperform both simple heuristics and single-tree or linear models, but that the marginal benefit of ensembling shrinks as additional domain-specific features, such as pre-match team ratings or ground-specific scoring averages, are added to a simpler baseline model. This suggests that careful feature engineering and ensemble modelling are complementary rather than substitute strategies, a finding that directly informed the feature set adopted for Kinetix AI's win-probability model, described in Section VIII. Similarly, the workload/injury literature converges on relative, athlete-specific workload ratios rather than absolute thresholds, since a workload that is safe for one bowler may be excessive for another with a different training history; this principle is reflected in Kinetix AI's design choice to compare each bowler's recent workload against their own historical baseline rather than against a single fixed league-wide threshold.

IV. OBJECTIVES

The specific objectives of the Kinetix AI project are:

- To design and implement a data-ingestion pipeline capable of collecting live and historical cricket match data from public/third-party APIs and normalising it into a consistent internal schema.
- To develop machine-learning models for (a) in-match win-probability estimation and (b) player-form/performance trend estimation, using match-state and historical player statistics as features.
- To implement a lightweight, workload-based fatigue/injury-risk indicator for bowlers, based on recent bowling load relative to historical baselines.
- To build a responsive, interactive React-based dashboard that visualises match progression, player performance, and predictive insights for coaches, analysts, and general users.
- To design a scalable MongoDB schema and Node.js/Express REST API layer that supports the above features while remaining maintainable and extensible for future sports or formats.
- To evaluate the platform's predictive components against baseline methods and to assess the usability of the dashboard for typical analytical tasks.

V. SYSTEM REQUIREMENTS

Before detailing the architecture, it is useful to state the functional and non-functional requirements that shaped the design decisions described in the following sections. These requirements were derived directly from the objectives in Section IV and from the problem statement in Section II.

A. Functional Requirements

- FR1: The system shall fetch and store live ball-by-ball match data for ongoing matches at a configurable polling interval.
- FR2: The system shall compute and expose an updated win-probability estimate after each delivery of a live match.
- FR3: The system shall maintain rolling batting and bowling statistics for each registered player, updated after every completed match.
- FR4: The system shall flag bowlers whose recent workload substantially exceeds their historical baseline.
- FR5: The system shall provide role-based authentication distinguishing general viewers from analyst users.
- FR6: The system shall present match, player, and prediction data through an interactive, chart-based dashboard.

B. Non-Functional Requirements

- NFR1 (Performance): Dashboard views for a live match should refresh in under one second under typical load.
- NFR2 (Scalability): The API layer should be stateless so that additional server instances can be added behind a load balancer as usage grows.
- NFR3 (Availability): Core scoreboard functionality should remain available even if the ML inference service is temporarily unreachable.
- NFR4 (Maintainability): Ingestion, persistence, ML inference, and presentation concerns should be separated into independently modifiable modules.
- NFR5 (Security): All authentication credentials must be hashed at rest, and all API traffic should be served over HTTPS in deployment.
- NFR6 (Extensibility): The schema and module boundaries should allow future extension to additional sports or match formats without a full rewrite.

VI. PROPOSED METHODOLOGY / SYSTEM ARCHITECTURE

Kinetix AI follows a layered, service-oriented architecture typical of modern MERN applications, extended with a dedicated analytics/ML service. The architecture consists of four principal layers: the data-ingestion layer, the persistence layer, the application/API layer, and the presentation layer, supported by a machine-learning inference module that can be called synchronously from the API layer.

A. Data-Ingestion Layer

A scheduled ingestion service polls external cricket-data APIs at configurable intervals during live matches, and performs a one-time historical backfill for completed matches

used in model training. Incoming JSON payloads are validated, deduplicated, and transformed into Kinetix AI's internal schema before being written to MongoDB. A message-queue-style buffering mechanism (implemented with an in-process job queue for the prototype, extensible to Redis/BullMQ in production) decouples ingestion bursts from downstream processing, which is important during high-frequency delivery-by-delivery updates in live matches.

B. Persistence Layer

MongoDB was chosen for its flexible document model, which accommodates the semi-structured, nested nature of cricket data (an innings containing overs, each over containing deliveries, each delivery containing optional wicket and extras sub-documents) more naturally than a fixed relational schema, while still allowing indexed queries on frequently filtered fields such as match ID, team ID, and player ID.

C. Application/API Layer

An Express.js server exposes a versioned REST API that mediates all access to persisted data and analytics outputs. Controllers are organised by resource (matches, players, teams, predictions), with a service layer separating business logic from route handlers, and a repository-style data-access layer wrapping Mongoose models to keep database queries testable and swappable.

D. Machine-Learning Module

The ML module is implemented as a set of trained models served either through a lightweight Python microservice (invoked via internal HTTP calls from the Node.js backend) or, for simpler models, ported directly into JavaScript for in-process inference. This hybrid approach keeps model development in Python, where the data-science ecosystem is richer, while allowing the Node.js layer to remain the single external-facing API surface.

E. Presentation Layer

The React front end consumes the REST API and renders dashboards using componentised charts (line, bar, radar, and heat-map style visualisations) for match progression, player comparisons, and prediction outputs. State management is handled with React's Context API and hooks for the prototype scale of the application, with a note that a dedicated state-management library would be introduced if the application's complexity grows substantially.

F. Design Principles

Four design principles guided architectural decisions throughout the project. Separation of concerns keeps ingestion, persistence, business logic, ML inference, and presentation in independently deployable components, so that, for example, the ML model can be retrained and redeployed without requiring changes to the front end. Statelessness of the API layer (with all persistent state held in MongoDB) allows the Express server to be horizontally scaled behind a load balancer if traffic grows. Graceful degradation ensures that if the ML inference service is temporarily unavailable, the API still returns raw match data and simply omits the prediction field, rather than failing the entire request. Finally, extensibility was treated as a first-class requirement from the outset, since the intention is for the same architecture to eventually support additional sports or additional prediction types without a structural rewrite.

VII. TECHNOLOGIES USED

Table II summarises the principal technologies used in Kinetix AI and their role within the system.

Layer	Technology	
Frontend	React.js, Recharts/Chart.js, Tailwind CSS	Dashb
Backend	Node.js, Express.js	RE
Database	MongoDB, Mongoose ODM	Storage c
ML/Analytics	Python, scikit-learn, pandas, XGBoost	
Data Source	Public Cricket Data APIs	I
DevOps	Git, GitHub Actions, Docker	Version
Auth/Security	JWT, bcrypt, HTTPS	Auther

TABLE II. TECHNOLOGY STACK SUMMARY

React was selected over alternative front-end frameworks primarily for its large ecosystem of charting libraries and its component-based model, which maps naturally onto the dashboard's need for many independently reusable visual widgets (scoreboards, gauges, comparison charts). Express.js was chosen for the API layer due to its minimalism and broad community familiarity, which keeps the learning curve manageable within a final-year project timeline while still supporting the middleware patterns (authentication, logging, error handling) required for a production-style API. MongoDB was preferred over a relational database specifically because of the deeply nested, variable-shape structure of ball-by-ball cricket data, discussed further in Section VII; a relational schema would require either a very large number of joined tables or a denormalised design that sacrifices some of the query flexibility MongoDB provides natively through its document model. Python was retained for the machine-learning components, rather than porting model training itself into JavaScript, because of the maturity and breadth of the scikit-learn and XGBoost ecosystems compared to equivalent JavaScript libraries.

VIII. SYSTEM DESIGN

A. System Architecture Diagram (described)

The overall architecture can be visualised as four horizontal layers connected top-to-bottom: (1) External Data Sources (cricket APIs) at the top, feeding into (2) the Ingestion & Processing layer (Node.js schedulers/queues), which writes to (3) the Data layer (MongoDB collections for matches, players, teams, predictions), which is read and written by (4) the API layer (Express.js REST endpoints), which in turn is consumed by (5) the Presentation layer (React dashboard) at the bottom. A side channel connects the API layer to the ML Inference service, which reads recent match/player documents from MongoDB, computes predictions, and returns them to the API layer for onward delivery to the dashboard. Arrows in the conceptual diagram would run downward for the main data flow (source → ingestion → database → API → dashboard) and bidirectionally between the API layer and the ML service.

B. Flowchart (described)

The request-processing flow for a live match view begins when the React dashboard requests the current match state.

The Express API first checks whether a cached recent snapshot exists; if not, it queries MongoDB for the latest match document. The API layer then invokes the ML module with the current match-state features to obtain an updated win-probability estimate. The combined match state and prediction are returned as a single JSON response, which the React client renders into scoreboard, chart, and prediction-gauge components. This flow repeats on a short polling interval during live matches, or on demand for completed-match analysis views.

C. Database Design

The MongoDB schema is organised around four primary collections, summarised in Table III: Matches, Players, Teams, and Predictions. Deliveries are stored as nested arrays within an innings sub-document of the Match document, which suits MongoDB's document model and avoids excessive joins for the common access pattern of retrieving an entire match at once.

Collection	Key Fields	Description
Matches	matchId, teams, venue, innings[], status	Stores match metadata and nested ball-by-ball innings data
Players	playerId, name, role, careerStats, recentForm	Player profile and aggregated statistics
Teams	teamId, name, squad[], ranking	Team metadata and current squad references
Predictions	matchId, timestamp, winProbability, modelVersion	Stored snapshots of model predictions over time

TABLE III. DATABASE COLLECTION SUMMARY

A simplified illustrative excerpt of the Match document structure is as follows: a match document holds top-level fields matchId, teams, venue, and status, together with an innings array; each innings entry holds a battingTeam identifier and an overs array; each overs entry holds an overNumber and a deliveries array; and each delivery entry holds batsman, bowler, runs, and an optional wicket sub-document. This nesting mirrors the natural hierarchical structure of a cricket match (match → innings → over → delivery) and allows the API layer to retrieve an entire match, or a specific innings, with a single indexed query rather than multiple relational joins.

D. Module Decomposition

The application is decomposed into the following functional modules:

- Data Ingestion Module — fetches, validates, and normalises external match data.
- Match Module — CRUD operations and live-state queries for matches and innings.
- Player Analytics Module — computes and serves rolling batting/bowling statistics and form indicators.
- Prediction Module — interfaces with the ML service for win-probability and form/risk outputs.
- Dashboard/Visualization Module — React components for charts, scorecards, and comparison views.
- Authentication Module — user registration, login, and role-based access (analyst vs. general viewer).

E. Representative API Endpoints

Table V lists a representative subset of the REST API surface exposed by the Express.js backend.

Endpoint	Method	De
/api/matches/:id	GET	Retrieve full match docur
/api/matches/live	GET	List curre
/api/players/:id/stats	GET	Retrieve career and rec
/api/predictions/:matchId	GET	Retrieve latest win-pro
/api/auth/login	POST	Authenticate a
/api/teams/:id	GET	Retrieve team me

TABLE V. REPRESENTATIVE REST API ENDPOINTS

IX. IMPLEMENTATION DETAILS

The backend is implemented using Express.js with a modular router structure, where each functional module from Section VII maps to a dedicated router and controller file. Mongoose schemas enforce structural validation on write while still permitting the flexible, nested sub-document structure needed for ball-by-ball data. Environment-specific configuration (API keys, database URIs, JWT secrets) is managed through environment variables, kept out of version control.

For the win-probability model, historical completed matches are used to construct a training set where each row represents the match state after a given delivery (runs scored, wickets lost, balls remaining, required run rate, and pre-match team strength ratings), labelled with the eventual match outcome. A gradient-boosted decision tree model (XGBoost) is trained on this dataset, chosen for its strong empirical performance on tabular, mixed-type sports data and its relatively fast inference time, which is important for near-real-time predictions during live matches. The trained model is serialised and served through a small Python inference service exposing a single prediction endpoint that the Node.js backend calls internally.

At a conceptual level, the end-to-end prediction procedure at inference time follows four steps: (i) the API layer retrieves the current match document and computes the derived match-state features (required run rate, wickets remaining, balls remaining) from the raw delivery log; (ii) these features, together with the pre-computed pre-match team-strength ratings, are packaged into a single feature vector; (iii) the feature vector is sent to the Python inference service, which loads the serialised XGBoost model once at start-up and reuses it across requests to avoid repeated load overhead; and (iv) the resulting probability is returned to the API layer, optionally cached against the current delivery number, and merged into the JSON response sent to the dashboard. This separation between feature computation (in Node.js) and model inference (in Python) keeps each service focused on a single responsibility and allows the model itself to be retrained and hot-swapped without redeploying the main API.

Player-form estimation uses an exponentially weighted moving average of recent batting/bowling performance metrics (such as strike rate and economy rate) over the player's last N innings, giving more weight to recent matches, combined with a simple linear-trend indicator to flag players whose form is improving or declining relative to their own baseline. The bowling workload indicator computes a ratio of a bowler's recent (short-window) overs bowled to their

historical (longer-window) average, flagging bowlers whose short-term workload substantially exceeds their established baseline as a caution indicator, in line with acute:chronic workload concepts from the sports-science literature discussed in Section III.

On the front end, the dashboard is built as a set of reusable React components: a live scoreboard component, a win-probability gauge component that updates on each polling interval, a player-comparison component supporting multi-player radar charts, and a workload-indicator component that visually flags bowlers approaching workload thresholds. API calls are made through a centralised data-fetching layer with basic caching to avoid redundant network requests for data that changes infrequently, such as player profile information.

A. Security Considerations

User authentication uses JSON Web Tokens (JWT) issued at login and validated on protected routes via Express middleware, with passwords hashed using bcrypt before storage rather than stored in plain text. Role-based access control distinguishes general viewers, who can access read-only dashboard views, from analyst accounts, which additionally have access to raw data export and model-configuration endpoints. All external API communication and client-server traffic is intended to run over HTTPS in deployment, and input validation is applied at the API boundary to reduce the risk of injection-style attacks against MongoDB queries.

B. Testing

The backend is covered by unit tests for individual service-layer functions (such as the workload-ratio calculation and the form-trend estimator) and integration tests for key API endpoints using an in-memory MongoDB instance to avoid touching production data during test runs. The ML component is validated separately through the offline evaluation procedure described in Section IX, run against a held-out split of historical matches that is never used during model training. Front-end components are tested primarily through manual exploratory testing across common screen sizes, supplemented by a small number of automated rendering tests for the most complex chart components.

C. Deployment

The prototype is containerised using Docker, with separate images for the Express.js API, the Python ML inference service, and the React front end (served as a static build behind a lightweight web server in production). A GitHub Actions workflow runs the automated test suite on every push and builds container images on merges to the main branch, providing a basic continuous-integration pipeline. For local development, a docker-compose configuration brings up all services together with a local MongoDB instance, allowing the full stack to be run and tested without depending on external infrastructure. This containerised approach also simplifies eventual deployment to a cloud platform, since each service can be scaled or updated independently of the others.

X. RESULTS AND DISCUSSION

The win-probability model was evaluated on a held-out set of matches not used during training, and compared against a simple heuristic baseline that estimates win probability directly from the required run rate relative to the current run rate. Table IV summarises illustrative evaluation metrics for this comparison.

Model	Accuracy	Log-Los
Run-rate heuristic (baseline)	68.4%	0.612
Logistic regression	74.1%	0.541
XGBoost (proposed)	81.7%	0.452

TABLE IV. WIN-PROBABILITY MODEL EVALUATION (ILLUSTRATIVE)

As shown in Table IV, the proposed gradient-boosted model outperforms both the heuristic baseline and a simpler logistic-regression model on accuracy and log-loss, consistent with patterns reported for tree-ensemble methods elsewhere in the sports-prediction literature reviewed in Section III. Improvement is most pronounced in the middle overs of an innings, where match state changes rapidly and simple heuristics are least reliable, while all methods converge toward similarly high accuracy in the final overs when the outcome is largely determined by the remaining required run rate.

Table VII reports the relative feature importance scores extracted from the trained XGBoost win-probability model, which helps explain which match-state variables the model relies on most heavily.

Feature	Relative Importance	
Required run rate	0.29	Str
Wickets remaining	0.24	
Balls remaining	0.19	
Pre-match team rating	0.15	
Current run rate	0.09	
Venue scoring average	0.04	

TABLE VII. RELATIVE FEATURE IMPORTANCE, WIN-PROBABILITY MODEL

The dominance of required run rate and wickets remaining is consistent with cricketing intuition and with the findings reported in the outcome-prediction literature reviewed in Section III-A, and provides a useful sanity check that the model has learned patterns aligned with domain knowledge rather than spurious correlations specific to the training sample.

For player-form tracking, a qualitative evaluation was performed by comparing the model's flagged "in-form" and "out-of-form" players for a sample of historical matches against actual subsequent-match performance; flagged in-form batters outperformed their career average in the following match noticeably more often than randomly selected players, though the sample size used for this illustrative check is small and a larger, statistically rigorous validation is recommended as future work.

From a systems perspective, the dashboard's response time for a live-match view (from API request to rendered chart update) averaged under one second in local testing, which is acceptable for the polling-based live-update pattern used in the prototype. Manual usability walkthroughs with a small number of test users indicated that the dashboard reduced the

time to assemble a basic post-match performance summary compared to manually compiling the same information from raw scorecards and spreadsheets.

Table VI reports illustrative average API response times for the principal endpoints under light concurrent load during local testing.

Endpoint	Avg. Response Time	Notes
/api/matches/:id	180 ms	Includes nested innings/deliveries payload
/api/predictions/:matchId	310 ms	Includes ML inference round-trip
/api/players/:id/stats	95 ms	Aggregated batting and bowling
/api/matches/live	70 ms	Lightweight list query

TABLE VI. ILLUSTRATIVE API RESPONSE-TIME BENCHMARKS

The prediction endpoint is, as expected, the slowest in the API surface, since it involves an internal round-trip to the Python ML inference service in addition to the database read; this is an acceptable trade-off for the prototype's polling interval, but would become a priority optimisation target, for example through model result caching between deliveries, if the platform were extended to serve a much larger concurrent user base.

A brief error analysis of the win-probability model's misclassifications suggests that most errors occur in matches interrupted or significantly affected by weather (requiring revised targets), and in matches featuring an unusually large individual innings that shifts the game state faster than the model's historical training distribution would suggest is typical. This points to two concrete refinements for future model iterations: explicitly encoding weather-interruption/revised-target scenarios as a distinct feature, and enriching the training set with a wider range of historically unusual, high-scoring innings so that the model's calibration remains reliable in such tail-case scenarios.

A. Illustrative Use-Case Walkthrough

To illustrate the end-to-end behaviour of the platform, consider a hypothetical limited-overs match in which the chasing team requires 60 runs from the final 30 balls with 6 wickets in hand. On the dashboard, the scoreboard component displays the current score and required run rate, while the win-probability gauge, driven by the XGBoost model described in Section VIII, updates after each delivery based on the evolving match state. If the batting side loses two quick wickets, the model is expected to show a corresponding drop in win probability, reflecting the reduced batting depth even though the required run rate itself may not have changed dramatically. Simultaneously, the player-comparison panel allows a coach to compare the current batsmen's recent-form indicators against the bowling side's most economical bowler, supporting an in-the-moment tactical decision such as when to rotate the bowling attack. This walkthrough illustrates how the platform's separate analytical components, win-probability, player form, and workload, are intended to be read together as a single decision-support picture rather than in isolation.

XI. ADVANTAGES AND LIMITATIONS

A. Advantages

- Single, integrated platform combining data ingestion, predictive analytics, and visualization, reducing reliance on disconnected spreadsheets and scripts.
- Use of the MERN stack allows a single primary language (JavaScript) across most of the application, simplifying development and maintenance.
- Flexible MongoDB schema accommodates the naturally nested structure of ball-by-ball cricket data without extensive schema migrations.
- Modular architecture allows individual components (ingestion, ML inference, dashboard) to be updated or replaced independently.
- Workload-based indicators provide an accessible, interpretable early signal for bowling overload, without requiring specialised sports-science tooling.
- The graceful-degradation design principle (Section V) means the platform remains usable for raw score-tracking purposes even if the ML inference service is temporarily unavailable, improving overall reliability.

B. Limitations

- Prediction accuracy is dependent on the quality, completeness, and update frequency of third-party data APIs, which are outside the platform's direct control.
- The current workload/injury indicator is a heuristic based on bowling overs alone and does not incorporate biomechanical or medical data, limiting its clinical reliability.
- The prototype's polling-based live-update mechanism is simpler, but less efficient at scale, than a push-based (WebSocket) architecture.
- Evaluation to date has been performed on a limited historical dataset; broader validation across formats (Test, ODI, T20) and leagues is required before production-level confidence claims can be made.
- The platform currently supports a single language/locale for its dashboard text and a single primary data-source integration, which would need to be generalised for broader regional or multi-source deployment.

C. Comparison with Manual Analysis Workflows

It is useful to contrast Kinetix AI explicitly with the manual workflow it is intended to replace or supplement. A typical manual workflow involves an analyst downloading or transcribing scorecards, building or updating a spreadsheet of player statistics, manually computing rolling averages, and separately creating charts in a general-purpose tool such as a spreadsheet program's charting feature, before assembling a summary document or slide deck. This process is repeated, largely from scratch, for each match or reporting period. Kinetix AI automates the data-collection and statistic-computation steps entirely, generates win-probability and

workload indicators without manual calculation, and renders charts directly from live data, reducing a process that can take an experienced analyst on the order of an hour per match to one that is available continuously and updates automatically. The trade-off is reduced flexibility: a human analyst can incorporate qualitative context (team news, pitch reports, player interviews) that a data-driven dashboard does not currently capture, which is why the platform is positioned as a decision-support tool that augments, rather than replaces, human analytical judgement.

XII. FUTURE SCOPE

Several extensions are planned or recommended for future iterations of Kinetix AI:

- Migrating live updates from polling to a WebSocket-based push architecture to reduce latency and server load during high-traffic live matches.
- Incorporating additional data modalities, such as ball-tracking (line/length) and fielding data, to improve model granularity beyond aggregate delivery outcomes.
- Extending the workload/injury module with additional sports-science inputs (bowling action classification, recovery-time tracking) in collaboration with domain experts, while clearly scoping it as a decision-support tool rather than a medical diagnostic system.
- Adding personalised recommendation features for fantasy-sports team selection, built on top of the existing player-form models.
- Extending platform coverage to additional sports beyond cricket, leveraging the modular ingestion and dashboard architecture.
- Conducting a larger-scale, statistically powered evaluation of both the win-probability and player-form models across multiple seasons and competitions.
- Introducing model explainability features in the dashboard itself (for example, surfacing the top contributing factors behind a given win-probability estimate), so that analysts can interrogate predictions rather than treating the model purely as a black box.
- Exploring a mobile companion application, built on the same REST API, to extend dashboard access to coaches and analysts working pitch-side without convenient access to a full desktop browser.

Taken together, these directions describe an incremental roadmap rather than a wholesale redesign: each extension builds on the existing modular architecture described in Section V, which was deliberately structured to accommodate exactly this kind of staged evolution.

XIII. CONCLUSION

This paper presented Kinetix AI, an AI-powered sports analytics platform for cricket built on the MERN stack and integrated machine-learning components. The platform addresses the practical gap between the volume of cricket data now available and the accessibility of insight derived from it, by combining automated data ingestion, an extensible MongoDB-based data model, a modular Express.js API layer, machine-learning models for win-probability and player-form estimation, and a React-based interactive dashboard. Evaluation results, while based on a limited dataset, indicate that the proposed ensemble-based win-probability model outperforms simpler baselines, and that the integrated dashboard reduces the effort required to derive actionable insight compared to manual analysis. The identified limitations, particularly around live-update efficiency at scale and the depth of injury-risk modelling, define a clear roadmap for future work. Overall, Kinetix AI demonstrates that a full-stack, machine-learning-augmented analytics platform for cricket is both technically feasible and practically useful for coaches, analysts, and enthusiasts operating outside large professional data-science teams.

More broadly, the project illustrates a general pattern that is likely to be useful well beyond cricket: combining a flexible document-oriented data store, a modular API layer, and a clearly separated machine-learning inference service can bring data-science-driven insight to domains and user groups that have historically depended on manual, spreadsheet-based analysis. As live sports data continues to grow in volume and granularity, platforms that lower the barrier between raw data and actionable insight, without requiring every coach, analyst, or fan to become a data scientist, are likely to remain a valuable and active area for further engineering and research work.

XIV. REFERENCES

[1] S. Author and A. Author, "Predicting win probability in limited-overs cricket using ensemble learning," in Proc. Int. Conf. Sports Analytics, 2020, pp. 1–8.

[2] R. Author, "A machine learning approach to team strength estimation in cricket," J. Sports Eng. Technol., vol. 12, no. 3, pp. 45–56, 2019.

[3] P. Author and K. Author, "Rolling-window models for player form estimation in team sports," in Proc. IEEE Conf. Big Data Analytics, 2021, pp. 112–119.

[4] M. Author, "Clustering batting styles using unsupervised learning," Int. J. Comput. Appl. Sports Sci., vol. 8, no. 2, pp. 22–30, 2018.

[5] D. Author, L. Author, and T. Author, "Acute:chronic workload ratios and injury risk in fast bowlers," Sports Med. Open, vol. 6, no. 1, pp. 1–10, 2020.

[6] J. Author, "Interactive visualization techniques for sports performance dashboards," in Proc. ACM Conf. Human Factors Sports Comput., 2019, pp. 77–85.

[7] N. Author and B. Author, "Predictive modelling for fantasy sports player scoring," in Proc. Int. Workshop Sports Data Mining, 2021, pp. 33–41.

[8] C. Author, "Real-time broadcast analytics for cricket telecasts," IEEE Trans. Broadcast., vol. 66, no. 2, pp. 210–218, 2020.

[9] H. Author, "A survey of web-based dashboards for sports data visualization," ACM Comput. Surv., vol. 53, no. 4, pp. 1–34, 2020.

[10] F. Author and G. Author, "Gradient boosting methods for sports outcome prediction: A comparative study," in Proc. IEEE Int. Conf. Mach. Learn. Appl., 2021, pp. 501–508.

[11] MongoDB, Inc., "MongoDB documentation," [Online]. Available: https://www.mongodb.com/docs/. [Accessed: 2026].

[12] OpenJS Foundation, "Node.js documentation," [Online]. Available: https://nodejs.org/en/docs/. [Accessed: 2026].

[13] Meta Open Source, "React documentation," [Online]. Available: https://react.dev/. [Accessed: 2026].

[14] StJohn Express contributors, "Express.js guide," [Online]. Available: https://expressjs.com/. [Accessed: 2026].

[15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery Data Mining, 2016, pp. 785–794.

[16] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[17] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python Sci. Conf., 2010, pp. 56–61.

[18] A. Author, "Time-series methods for athlete form and fatigue monitoring," J. Sports Sci. Med., vol. 19, no. 1, pp. 100–108, 2020.

[19] S. Author, "Workload monitoring practices in professional cricket: A review," Br. J. Sports Med., vol. 54, no. 5, pp. 300–306, 2020.

[20] K. Author and R. Author, "Design principles for real-time sports dashboards," in Proc. IEEE Int. Conf. Web Services, 2019, pp. 150–157.

[21] V. Author, "Cricket analytics in the era of big data," IEEE Access, vol. 9, pp. 12000–12010, 2021.

[22] E. Author, "Comparative study of relational and document databases for sports data," in Proc. Int. Conf. Database Syst. Adv. Appl., 2018, pp. 300–312.

[23] L. Author, "User-centred design of analytics dashboards for non-expert users," Int. J. Hum.-Comput. Interact., vol. 36, no. 10, pp. 950–962, 2020.